

This is a comprehensive orchestration system for building a Linux distribution from scratch. The architecture is thoughtful, but there are several critical issues that will likely cause failures in real-world use. I'll address your specific concerns first, then provide general observations.

1. Shell Scripting Safety

Status: Several issues with quoting, word splitting, and set -e behavior

Unquoted Expansions

The scripts have many unquoted variable expansions that will break with spaces or special characters:

```
# In build-intergenos.sh
su - "$BUILD_USER" -c "env -i HOME=/home/${BUILD_USER} TERM=${TERM} bash
${SCRIPTS}/toolchain-build.sh" 2>&1 | tee -a "$BUILD_LOG"
```

The `TERM=${TERM}` expansion is okay (`TERM` shouldn't have spaces), but `${SCRIPTS}` and `$BUILD_LOG` are unquoted in the tee redirection. Actually `tee -a "$BUILD_LOG"` is quoted — good. But the `su` command string itself is problematic.

More concerning:

```
# In chroot-build-ch8.sh
tar -xf "${IGOS_SOURCES}/${tarball}" -C "$workdir" --strip-components=1 || {
```

This is correctly quoted. However, in `chroot-build-core-extra.sh`:

```
PYYAML_TAR=$(ls ${IGOS_SOURCES}/PyYAML-*.tar.gz ${IGOS_SOURCES}/pyyaml-*.tar.gz
2>/dev/null | head -1)
```

The `ls` command has unquoted globs. If any filename contains spaces (unlikely for tarballs, but still), this will break. Use an array:

```
files=("$IGOS_SOURCES"/PyYAML-*.tar.gz "$IGOS_SOURCES"/pyyaml-*.tar.gz)
PYYAML_TAR="${files[0]}"
```

set -e Pitfalls in Function Contexts

The scripts use set -e (errexit) but also have command substitutions and conditionals that can behave unexpectedly:

```
# In pkg-functions.sh
if declare -f do_install > /dev/null 2>&1; then
    echo "=== [INSTALL] $(date '+%Y-%m-%d %H:%M:%S') ===" >> "$install_log"
    do_install >> "$install_log" 2>&1
else
    pkg_error "No do_install() function defined for ${name}"
    return 1
fi
```

If do_install fails, set -e would normally exit the script, but because it's inside an if condition (the function call is in the then-block, not the condition), set -e does **NOT** apply. The || after the function call is correct, but the return 1 only triggers if declare -f fails, not if do_install fails.

Fix: Check the exit code explicitly:

```
if declare -f do_install > /dev/null 2>&1; then
    echo "=== [INSTALL] $(date '+%Y-%m-%d %H:%M:%S') ===" >> "$install_log"
    if ! do_install >> "$install_log" 2>&1; then
        pkg_error "do_install failed for ${name}"
        return 1
    fi
fi
```

Word Splitting in pkg_manifest

```
file_list=$(cd "$dest" && find . -mindepth 1 | sed 's|^\./||' | sort)
```

This is captured in a variable, then used in a here-doc. That's safe because it's not split. But later:

```
local file_count
file_count=$(find "$PKG_DEST" -not -type d | wc -l)
```

The find output is piped to wc -l — that's fine.

Missing set -o pipefail

Many scripts use set -e but not set -o pipefail. This means failures in the middle of a pipeline can be ignored:

```
python3 igos-build.py ... 2>&1 | tee -a "$LOG"
BUILD_RC=${PIPESTATUS[0]}
```

This correctly uses PIPESTATUS, but other pipelines don't. For example:

```
tail -20 "$pkg_log" | while IFS= read -r l; do log "  $l"; done
```

If tail fails, the pipeline continues. Add set -o pipefail to all scripts.

umask 022 at Script Start

This is correct for LFS builds.

2. Chroot Mount/Unmount Correctness

Status: Generally correct but has critical issues

Mount Order Is Wrong

In chroot-setup.sh:

```
mount -v --bind /dev $IGOS/dev
mount -vt devpts devpts -o gid=5,mode=0620 $IGOS/dev/pts
mount -vt proc proc $IGOS/proc
```

```
mount -vt sysfs sysfs $IGOS/sys
mount -vt tmpfs tmpfs $IGOS/run
```

The order is correct (dev before dev/pts). But chroot-teardown.sh has:

```
umount $IGOS/dev/shm 2>/dev/null
umount $IGOS/run 2>/dev/null
umount $IGOS/sys 2>/dev/null
umount $IGOS/proc 2>/dev/null
umount $IGOS/dev/pts 2>/dev/null
umount $IGOS/dev 2>/dev/null
```

This is **WRONG ORDER**. /dev/pts must be unmounted before /dev. The script does dev/pts then dev, which is correct, but sys and proc order relative to each other doesn't matter. However, run should be unmounted before proc/sys? Actually run is independent.

The real problem: /dev/shm is unmounted first, but it's a subdirectory of /dev. That's fine because it's a separate tmpfs mount.

More serious: The teardown script never unmounts /dev/shm if it's a symlink case. In chroot-setup.sh:

```
if [ -h $IGOS/dev/shm ]; then
    install -v -d -m 1777 $IGOS$(realpath /dev/shm)
else
    mount -vt tmpfs -o nosuid,nodev tmpfs $IGOS/dev/shm
fi
```

If it's a symlink, no mount is created, so unmounting is correct. But the symlink target directory is created but never cleaned up. Not a big issue.

Missing --make-rslave for Bind Mounts

In the chroot, systemd expects mounts to be slave mounts. Without this, unmounting can fail. Add:

```
mount --make-rslave $IGOS/dev
mount --make-rslave $IGOS/proc
mount --make-rslave $IGOS/sys
```

No Check for Stale Mounts Before Setup

If a previous build crashed, mounts might still be present. `chroot-setup.sh` checks `mountpoint -q` before mounting, which is good. But it doesn't attempt to clean stale mounts. Consider adding a force-cleanup option.

chroot-enter.sh Uses Wrong PATH

```
PATH=/usr/bin:/usr/sbin \
```

This lacks `/bin` and `/sbin` (which are symlinks to `/usr/bin` and `/usr/sbin` on modern systems, but during early `chroot`, those symlinks may not exist yet). Add `/bin` and `/sbin` for safety.

3. DESTDIR Staging and Deployment

Status: Well-implemented, but has edge case bugs

Root Symlink Collision Detection

In `pkg-functions.sh`:

```
for entry in lib lib64 bin sbin; do
    if [ -d "${dest}/${entry}" ] && [ ! -L "${dest}/${entry}" ] && [ -L "/"${entry} ]; then
        dangerous="${dangerous} ${entry}"
    fi
done
```

This correctly detects if a package is trying to create a real directory at a location that's a symlink on the live system. However:

Missing check for `/etc`: Some packages might install to `/etc` directly, but `/etc` is a real directory. That's fine. But if a package tries to install to `/usr/etc`? That's also fine.

Missing check for existing files: If a package tries to overwrite a file owned by another package, the manifest will show it later, but the deploy step will overwrite it. Consider adding a pre-deploy conflict check.

Tar Pipe Safety

```
tar -C "${dest}" -cf - . \
| tar -C / -xf - --no-overwrite-dir --keep-directory-symlink
```

The flags are well-chosen:

- no-overwrite-dir: Preserves metadata of existing directories
- keep-directory-symlink: Follows existing symlinks rather than replacing them

However, this doesn't protect against **symlink traversal** where a package contains a symlink pointing outside the staging area. Example: A package creates staging/etc/passwd -> /etc/passwd and then a file staging/etc/passwd/evil. When extracted, it would write through the symlink.

Fix: Validate symlinks before extraction:

```
# After extraction to staging, check for dangerous symlinks
while IFS= read -r link; do
    target=$(readlink "$link")
    if [[ "$target" = /* && ! "$target" =~ ^"$dest" ]]; then
        pkg_error "Dangerous symlink in package: $link -> $target"
        return 1
    fi
done < <(find "$dest" -type l)
```

Missing --xattrs Preservation

The tar pipe doesn't preserve extended attributes (SELinux, capabilities). Consider adding -xattrs if needed.

4. Checkpoint Save/Restore

Status: Flawed — will fail in practice

Checkpoint Saves Before Phase Completion

In build-intergenos.sh:

```
if $CHECKPOINT; then
  case "$phase" in
    toolchain|core|kernel|desktop)
      save_checkpoint "$phase"
    ;;
  esac
fi
```

The checkpoint is saved **after** the phase completes, which is correct. But save_checkpoint does:

```
# Tear down chroot mounts temporarily for a clean snapshot
bash "${SCRIPTS}/chroot-teardown.sh" > /dev/null 2>&1 || true
tar -C "$IGOS" --one-file-system -czf "$checkpoint" . 2>&1
# Re-mount chroot filesystems
bash "${SCRIPTS}/chroot-setup.sh" > /dev/null 2>&1 || true
```

Problems:

The teardown may fail if the chroot is still in use (e.g., a shell is cd'd into it). The script uses || true to ignore failures, but if teardown fails, the tar will include mounted virtual filesystems (because --one-file-system prevents descending into them, but the mount points themselves are empty directories — that's fine).

However, if teardown partially fails, the tar could include /proc/kcore etc.

Re-mount assumes the same state as initial setup, but if the user was in the middle of a build, some mounts might have been modified (e.g., --make-rslave). Re-running chroot-setup.sh might not restore the exact state.

No locking — if two processes try to checkpoint simultaneously, chaos.

Checkpoint directory is /home — This assumes the build user's home directory exists and has space. For a 500GB root filesystem, the checkpoint will be huge (tens of GB). Consider using a dedicated directory with space check.

Restore procedure is documented but not implemented — The user must manually restore. That's acceptable for a build system, but error-prone.

Recommended approach: Skip checkpoints for now, or implement them as optional and clearly document the limitations.

Phase Resume Uses Phase Name, Not Package Name

The `--start-at` flag uses phase names (toolchain, core, etc.), which is correct for the orchestrator. But the phase implementations also have internal resume logic:

```
# In chroot-build-ch8.sh
SKIP=true
if [ -z "$IGOS_START_AT" ]; then
    SKIP=false
fi
```

This uses `IGOS_START_AT` to skip packages within a phase. That's a good design. However, the orchestrator doesn't pass `IGOS_START_AT` into the chroot environment. In `chroot-enter.sh`:

```
IGOS_START_AT="${IGOS_START_AT:-}" \
```

It is passed! Good. But in `build-intergenos.sh`, `IGOS_START_AT` is never set from `--start-at` for internal phases. The `--start-at` only applies to top-level phases. If you resume at core, it will rebuild all core packages from the beginning because `IGOS_START_AT` is empty inside the chroot.

Fix: In `run_phase`, when starting at a phase, also set `IGOS_START_AT` to the first package in that phase (or to a special value like "start").

5. CWD Reset Between Build Phases

Status: Correct approach, but incomplete

In chroot-build-ch8.sh:

```
# --- CONFIGURE ---
if declare -f configure > /dev/null 2>&1; then
    cd "$workdir"
    configure >> "$pkg_log" 2>&1
    # ...
fi

# --- BUILD ---
if declare -f build > /dev/null 2>&1; then
    cd "$workdir"
    build >> "$pkg_log" 2>&1
    # ...
fi
```

This resets CWD before each phase, which is necessary because bash functions can `cd` and that change persists. However, there's a **race condition**: If a function `cds` into a subdirectory and doesn't return to the original directory, the next `cd "$workdir"` will still succeed because `$workdir` is absolute. That's fine.

But: The `post_install` function is run after `pkg_install`, and `pkg_install` may have changed the working directory. In `chroot-build-ch8.sh`:

```
cd "$workdir"
log " [INSTALL] staging..."
pkg_install "$name" "$version" "$description"
# ...

# --- POST-INSTALL (runs on live system if defined) ---
if declare -f post_install > /dev/null 2>&1; then
    cd "$workdir"
    post_install >> "$pkg_log" 2>&1
fi
```

The `cd "$workdir"` before `post_install` is correct.

Edge case: If `pkg_install` fails and exits early, the `cd` for `post_install` won't run. That's fine.

Missing reset in `pkg-functions.sh`: The `pkg_install` function calls `pkg_stage`, `pkg_manifest`, etc., but doesn't reset CWD. That's acceptable because it's not expected to change CWD.

6. Package Tracking Correctness

Status: Solid implementation with minor issues

Manifest Generation Uses `find` Without `-type`

```
file_list=$(cd "$dest" && find . -mindepth 1 | sed 's|^\./||' | sort)
```

This includes directories, files, and symlinks. That's correct for Slackware-style manifests. But directories are listed without trailing `/`, which makes it impossible to distinguish a directory from a file with the same name (unlikely). Add trailing slashes for directories:

```
file_list=$(cd "$dest" && find . -mindepth 1 -printf "%y %P\n" | awk '{if($1=="d") print $2"/"; else print $2}' | sort)
```

Archive Creation Doesn't Preserve Symlinks Correctly

```
tar -C "$dest" -czf "$archive" .
```

This preserves symlinks correctly because `tar` defaults to following symlinks only if `-h` is given. It's correct.

Deployment Doesn't Verify After Copy

The `pkg_deploy` function doesn't verify that files were correctly written. Consider adding a checksum comparison or at least checking that the file count matches.

pkg_remove Is Dangerous

```
pkg_remove() {  
    local files  
    files=$(pkg_files "$pkg" | sort -r)  
    while IFS= read -r file; do  
        rm -f "$fullpath"  
    done <<< "$files"  
}
```

This removes files without checking if they're modified or owned by another package. The package manager (pkm) has config file preservation, but these bash functions don't. For the build system (which only installs, never removes), that's fine. But document that `pkg_remove` is for emergency use only.

Missing Lock File

If two build processes run simultaneously (unlikely but possible), the package tracking system could corrupt manifests. Add a flock:

```
exec 200>/var/lock/igos-pkg.lock  
flock -n 200 || { pkg_error "Another build is running"; return 1; }
```

7. Signal Handling

Status: Good but incomplete

In `build-intergenos.sh`:

```
CURRENT_PHASE=""  
cleanup() {  
    log ""  
    log "!!! Build interrupted during phase: ${CURRENT_PHASE:-none}"  
    log "!!! Resume with: sudo bash $0 --user $BUILD_USER --start-at ${CURRENT_PHASE:-  
validate}"  
    log ""  
    exit 130  
}
```

```
}  
trap cleanup SIGINT SIGTERM
```

This is good — it prints the resume command. However:

It doesn't unmount chroot or clean up. The user will have to run `chroot-teardown.sh` manually before resuming. Document this.

It doesn't propagate the signal to child processes. If a build script is running, Ctrl+C will kill the orchestrator, but the child (e.g., `su` or `chroot`) may continue. Use `trap ... SIGINT` and then kill the process group.

No trap for SIGHUP (terminal disconnect). If running over SSH and the connection drops, the build continues in the background. Add `trap cleanup SIGHUP`.

Improved handler:

```
cleanup() {  
    local sig=$1  
    log "!!! Received signal $sig — terminating build"  
    # Kill entire process group  
    kill -TERM -$$ 2>/dev/null || true  
    # Give children time to exit  
    sleep 2  
    kill -KILL -$$ 2>/dev/null || true  
    exit 130  
}  
trap 'cleanup SIGINT' SIGINT  
trap 'cleanup SIGTERM' SIGTERM  
trap 'cleanup SIGHUP' SIGHUP
```

Additional Critical Issues

1. The `sync_chroot_scripts` Function Is Incomplete

In `build-intergenos.sh`:

```
sync_chroot_scripts() {  
    log " Syncing scripts into chroot..."
```

```

rsync -a /mnt/intergenos/scripts/ "$IGOS/mnt/intergenos/scripts/"
rsync -a /mnt/intergenos/packages/ "$IGOS/mnt/intergenos/packages/"
rsync -a /mnt/intergenos/config/ "$IGOS/mnt/intergenos/config/" 2>/dev/null || true
rsync -a /mnt/intergenos/igos-build.py "$IGOS/mnt/intergenos/" 2>/dev/null || true
rsync -a /mnt/intergenos/igos-build/ "$IGOS/mnt/intergenos/igos-build/" 2>/dev/null ||
true
}

```

This runs **inside the chroot**? No, it runs on the host, copying files into \$IGOS/mnt/intergenos/. That's correct because \$IGOS is the chroot root. But the function is called from the host, and \$IGOS is defined as /mnt/igos. Good.

Problem: The function runs rsync without checking if the source directories exist. If /mnt/intergenos/config/ doesn't exist, it fails silently (due to || true). That's fine, but the user won't know.

More serious: The packages directory is copied **every time** before a phase. For the desktop tier, that's 337 packages. Rsync will be fast, but if the build is interrupted and restarted, changes to package templates will be picked up — that's the intended behavior.

2. chroot-build-desktop.sh Sets PyYAML Incorrectly

```

PYYAML_TAR=$(ls ${IGOS_SOURCES}/PyYAML-*.tar.gz ${IGOS_SOURCES}/pyyaml-*.tar.gz
2>/dev/null | head -1)
if [ -z "$PYYAML_TAR" ]; then
    log "ERROR: No PyYAML tarball found in $IGOS_SOURCES"
    exit 1
fi

```

The desktop build requires PyYAML, but the core build didn't install it. The script tries to install it manually, but it's installing **into the host Python's site-packages**, not into a virtual environment. This will work, but if the system Python is upgraded, PyYAML will break.

Better approach: Install PyYAML via pip if available, or add it as a package in the core tier.

3. create-image.sh Assumes BIOS Boot Only

The script creates a BIOS-bootable image only (GPT + bios_grub). It doesn't support EFI. The installer (Forge) supports both, but the image creator doesn't.

Fix: Detect EFI or add a --efi flag.

4. Image Creation Uses NBD, Which May Not Be Available

qemu-nbd requires the nbd kernel module. In some environments (containers, WSL), this isn't available. Add a fallback using losetup or kpartx.

5. Hardcoded Passwords in Image

```
chroot "$MOUNT_POINT" /bin/bash -c '  
  chpasswd <<< "root:intergenos"  
  passwd -x 99999 root  
,
```

The image has a hardcoded root password (intergenos). For a development image, that's acceptable, but document it clearly. For production, the installer should set the password.

6. No Validation of \$IGOS Before Teardown

In chroot-teardown.sh:

```
umount $IGOS/dev/shm 2>/dev/null  
umount $IGOS/run 2>/dev/null
```

If \$IGOS is empty (e.g., due to a bug), this would try to unmount /dev/shm on the host system — catastrophic. Add a safety check:

```
if [ -z "$IGOS" ] || [ "$IGOS" = "/" ]; then  
  echo "ERROR: IGOS is empty or root — refusing to unmount"  
  exit 1  
fi
```

7. The pkg_functions.sh Uses \$IGOS_LOGS Without Default

In pkg_log:

```
pkg_log() {  
    echo "[pkg] $" | tee -a "$IGOS_LOGS/pkg-install.log"  
}
```

If IGOS_LOGS is not set, this will write to /pkg-install.log (root of filesystem). Set a default:

```
IGOS_LOGS="${IGOS_LOGS:-/var/log/igos-build}"
```

Summary Table

Area	Rating	Notes
Shell safety	6/10	Missing pipefail, unquoted expansions, set -e pitfalls
Chroot mounts	7/10	Order mostly correct; missing --make-rslave; no stale mount cleanup
DESTDIR staging	8/10	Symlink collision detection good; missing symlink traversal protection
Checkpoint save	3/10	Teardown unreliable; no locking; huge file sizes; restore not implemented
CWD reset	8/10	Correct approach; one missing case
Package tracking	7/10	Solid; missing manifest trailing slashes; no locking
Signal handling	6/10	Doesn't kill child processes; missing SIGHUP

Priority Fixes Before Use

Add safety check for \$IGOS in teardown scripts (critical — prevents host system unmount)

Fix checkpoint system or disable it until reliable (critical for resume functionality)

Add set -o pipefail to all scripts (high)

Fix the IGOS_START_AT propagation so internal phase resume works (high)

Add symlink validation in pkg_deploy (high security)

Fix mount order in chroot-teardown.sh (medium)

Add --make-rslave to chroot mounts (medium)

Remove hardcoded root password from create-image.sh or document clearly (medium)

Add default for IGOS_LOGS (low)

Add flock to package tracking (low)

Additional Recommendations

Use ShellCheck

Run all bash scripts through [ShellCheck](#) to catch quoting and edge case issues. Many of the issues I found would have been caught.

Add a --dry-run Mode

For critical operations (partitioning, chroot teardown), a dry-run mode would help debugging.

Centralize Configuration

Many scripts have hardcoded paths (/mnt/igos, /mnt/intergenos, /var/lib/igos). Consider a single configuration file sourced by all scripts.

Add Log Rotation

The build logs can grow very large (GB for 470 packages). Add log rotation or at least a cleanup option.

Document Assumptions

The scripts assume:

- The host is Ubuntu (or has bash, coreutils, parted, etc.)
- The chroot has no internet access (sources pre-staged)
- The user has sudo access
- The build VM has enough disk space (500GB+)

Document these in the README.

This is a sophisticated build system that clearly has had significant thought put into it. The issues I've found are fixable, and the overall architecture is sound. With the critical fixes (especially the teardown safety and checkpoint reliability), this system should be capable of building a complete Linux distribution.